Football Stat Tracker - Project Context for Claude Code

This is the document for the scratch branch

Date: February 26, 2026
Project Status: Backend foundation complete, ready for API endpoints
Developer: Lancem238

Project Overview

Name: StatTrack - Real-Time Football Player Statistics Tracker

Goal: Build a full-stack web application that provides real-time statistics, performance analytics, and comparison tools for professional football players across the Big 5 European leagues (Premier League, La Liga, Serie A, Bundesliga, Ligue 1).

Learning Objectives:

- Full-stack TypeScript development
- RESTful API design
- PostgreSQL database design
- Third-party API integration (API-Football)
- Authentication with JWT
- Data visualization
- Cloud deployment

Technology Stack

Backend (Currently Working On)

- **Runtime:** Node.js 20 LTS
- **Framework:** Express.js
- **Language:** TypeScript
- **ORM:** Prisma
- **Database:** PostgreSQL 18
- **Caching:** Redis (planned, not yet implemented)
- **Authentication:** JWT (planned)
- **API:** API-Football via RapidAPI

Frontend (Not Started Yet)

- **Framework:** React 18+ (Vite)
- **UI:** shadcn/ui + TailwindCSS
- **Charts:** Chart.js with react-chartjs-2
- **State:** React Query + Context API

Deployment (Planned)

- **Frontend:** Vercel
- **Backend:** Railway
- **Database:** Railway PostgreSQL
- **Redis:** Upstash

Project Structure

```
C:\Users\lance\football-stat-tracker\
├── backend/
│   ├── node_modules/          (gitignored)
│   ├── prisma/
│   │   ├── schema.prisma      ☑ COMPLETE - Full database schema
│   │   └── migrations/        ☑ COMPLETE - Initial migration applied
│   ├── src/
│   │   ├── config/            (empty - ready for config files)
│   │   ├── routes/            (empty - ready for API routes)
│   │   ├── controllers/       (empty - ready for controllers)
│   │   ├── services/          (empty - ready for services)
│   │   ├── middleware/        (empty - ready for middleware)
│   │   ├── utils/             (empty - ready for utilities)
│   │   └── index.ts           ☑ COMPLETE - Basic server running
│   ├── .env                   ☑ COMPLETE - Environment variables set
│   ├── package.json           ☑ COMPLETE - Dependencies installed
│   ├── tsconfig.json          ☑ COMPLETE - TypeScript configured
│   └── .gitignore             ☑ COMPLETE
├── frontend/                  (not created yet)
├── .gitignore                 ☑ COMPLETE
└── README.md                  (optional)
```

What's Been Completed ☑

1. Development Environment Setup

- ☑ Node.js and npm installed
- ☑ PostgreSQL 18 installed and running
- ☑ VS Code configured
- ☑ Git initialized and connected to GitHub
- ☑ Repository: https://github.com/Lancem238/football-stat-tracker

2. Backend Foundation

- ☑ Express.js server created and running on port 3000
- ☑ TypeScript configured
- ☑ Basic middleware (helmet, cors, express.json)
- ☑ Health check endpoint: `GET /health`

- ☑ Test endpoint: `GET /`

Test the server:

```
cd backend
npm run dev
# Visit: http://localhost:3000
# Should show: {"message": "StatTrack API is running!", "status": "success"}
```

3. Database Setup

- ☑ PostgreSQL database created: `stattrack`
- ☑ Prisma ORM initialized
- ☑ Complete database schema defined (8 tables)
- ☑ Prisma Client generated
- ☑ Initial migration applied successfully

Database Tables:

1. `leagues` - Football leagues (Premier League, La Liga, etc.)
2. `teams` - Football teams
3. `players` - Player profiles
4. `player_stats` - Season-level statistics
5. `matches` - Match information
6. `match_performances` - Per-match player statistics
7. `users` - User accounts
8. `user_favorites` - User watchlists

4. Environment Configuration

- ☑ `.env` file created with all necessary variables
- ☑ Database connection configured
- ☑ API-Football key obtained from RapidAPI

Environment Variables Set:

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://postgres:PASSWORD@localhost:5432/stattrack
JWT_SECRET=change-this-to-a-random-secret-key-later
RAPIDAPI_KEY=[obtained from RapidAPI]
```

5. Dependencies Installed

Core:

- express

- cors
- helmet
- dotenv

TypeScript:

- typescript
- @types/node
- @types/express
- @types/cors
- ts-node
- nodemon

Database:

- prisma
- @prisma/client

Additional:

- zod (validation)
- bcrypt (password hashing)
- jsonwebtoken (JWT)

Current Status: Ready for API Development

What Works Right Now

- [x] Server starts successfully
- [x] Database connection established
- [x] Prisma Client generated
- [x] All tables created in PostgreSQL
- [x] TypeScript compilation working
- [x] Hot reload with nodemon working

Test Commands

```
# Start development server
cd backend
npm run dev

# Generate Prisma Client
npx prisma generate

# View database in GUI
npx prisma studio

# Check database migrations
npx prisma migrate status
```

Next Steps: API Development (Phase 1)

Immediate Next Tasks

1. Create API Service for Football-API

File: `backend/src/services/footballApi.service.ts`

Purpose:

- Handle all calls to API-Football
- Implement rate limiting (100 requests/day free tier)
- Cache responses in memory/Redis
- Error handling for API failures

Key Functions Needed:

```
searchPlayers(query: string)
getPlayerById(id: number)
getPlayerStats(playerId: number, season: string)
getTeamById(id: number)
getLeagueStandings(leagueId: number, season: string)
```

2. Create Player Routes

File: `backend/src/routes/players.ts`

Endpoints to Create:

```
GET  /api/v1/players/search?q={query}      - Search players
GET  /api/v1/players/:id                   - Get player details
GET  /api/v1/players/:id/stats             - Get player statistics
GET  /api/v1/players/:id/matches           - Get match history
GET  /api/v1/players/compare?ids={ids}     - Compare players
```

3. Create Player Controller

File: `backend/src/controllers/players.controller.ts`

Purpose:

- Handle business logic for player endpoints
- Call Football API service
- Store/retrieve data from database
- Format responses

4. Set Up Caching (Optional but Recommended)

File: `backend/src/services/cache.service.ts`

Purpose:

- Cache API responses to avoid hitting rate limits
- Implement TTL (time to live) for different data types
- Initially use in-memory cache, later add Redis

5. Error Handling Middleware

File: `backend/src/middleware/errorHandler.ts`

Purpose:

- Centralized error handling
- Consistent error response format
- Log errors
- Handle API-Football errors gracefully

API-Football Integration Details

Authentication

```
const headers = {
  'x-rapidapi-key': process.env.RAPIDAPI_KEY,
  'x-rapidapi-host': 'v3.football.api-sports.io'
};
```

Base URL

```
https://v3.football.api-sports.io
```

Key Endpoints to Use

```
GET /players?search={name}&season=2024
GET /players?id={playerId}&season=2024
GET /teams?id={teamId}
GET /leagues?season=2024
GET /standings?season=2024&league={leagueId}
GET /fixtures?league={leagueId}&season=2024
```

Rate Limits

- **Free Tier:** 100 requests per day
- **Resets:** Midnight UTC
- **Strategy:** Aggressive caching, prioritize popular players

Sample Response Format

```
{
  "response": [
    {
      "player": {
        "id": 154,
        "name": "Cristiano Ronaldo",
        "firstname": "Cristiano",
        "lastname": "Ronaldo",
        "age": 39,
        "birth": {
          "date": "1985-02-05",
          "place": "Funchal",
          "country": "Portugal"
        },
        "nationality": "Portugal",
        "height": "187 cm",
        "weight": "83 kg",
        "photo": "https://..."
      },
      "statistics": [...]
    }
  ]
}
```

Important File Locations

Configuration Files

- Environment: `backend/.env`
- TypeScript: `backend/tsconfig.json`
- Prisma Schema: `backend/prisma/schema.prisma`
- Package: `backend/package.json`

Entry Point

- Main Server: `backend/src/index.ts`

Where to Create New Files

- Routes: `backend/src/routes/`
- Controllers: `backend/src/controllers/`
- Services: `backend/src/services/`
- Middleware: `backend/src/middleware/`

- Utilities: `backend/src/utils/`
- Types/Models: `backend/src/models/` (create this folder)

Development Workflow

Starting Development

```
cd C:\Users\lance\football-stat-tracker\backend
npm run dev
```

After Making Schema Changes

```
npx prisma generate
npx prisma migrate dev --name description_of_change
```

Checking Database

```
npx prisma studio
# Opens GUI at http://localhost:5555
```

Git Workflow

```
git status
git add .
git commit -m "Descriptive message"
git push origin main
```

Database Schema Reference

Key Tables and Their Purpose

leagues

- Stores football leagues (Premier League, La Liga, etc.)
- Fields: id, name, country, season, logoUrl

teams

- Stores team information
- Fields: id, name, leagueId, logoUrl, stadium, city
- Relations: belongs to league, has many players

players

- Stores player profiles
- Fields: id, name, teamId, position, nationality, birthDate, photoUrl, marketValue
- Relations: belongs to team, has many stats

player_stats

- Season-level statistics for each player
- Fields: playerId, season, goals, assists, appearances, rating
- One record per player per season per league

matches

- Match information
- Fields: id, homeTeamId, awayTeamId, date, homeScore, awayScore, status

match_performances

- Per-match player statistics
- Fields: matchId, playerId, goals, assists, rating, minutesPlayed

users

- User accounts for authentication
- Fields: email, username, passwordHash

user_favorites

- User watchlists
- Fields: userId, playerId, addedAt

API Response Format Standards

Success Response

```
{
  "success": true,
  "data": {
    // actual data here
  },
  "message": "Optional success message"
}
```

Error Response

```
{
  "success": false,
  "error": {
```

```
    "code": "ERROR_CODE",
    "message": "Human-readable error message"
  }
}
```

Pagination (for list endpoints)

```
{
  "success": true,
  "data": [...],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 100,
    "totalPages": 5
  }
}
```

Testing Endpoints

Using Browser

```
http://localhost:3000/
http://localhost:3000/health
```

Using curl

```
# Test basic endpoint
curl http://localhost:3000/

# Test with query params (once implemented)
curl "http://localhost:3000/api/v1/players/search?q=ronaldo"

# Test specific player (once implemented)
curl http://localhost:3000/api/v1/players/154
```

Using Postman/Thunder Client

1. Create a new request
2. Set URL: `http://localhost:3000/api/v1/players/search`
3. Add query param: `q=ronaldo`
4. Send request

Common Commands Reference

NPM Commands

```
npm run dev          # Start development server
npm run build        # Build for production
npm start            # Start production server
npm install <pkg>    # Install new package
```

Prisma Commands

```
npx prisma generate               # Generate Prisma Client
npx prisma migrate dev            # Create and apply migration
npx prisma migrate deploy         # Apply migrations (production)
npx prisma studio                 # Open database GUI
npx prisma db push                # Push schema without migration
npx prisma db pull                # Pull schema from database
```

Git Commands

```
git status                        # Check changes
git add .                         # Stage all changes
git commit -m "message"           # Commit changes
git push origin main              # Push to GitHub
git pull origin main              # Pull from GitHub
```

Known Issues and Gotchas

PowerShell vs Bash

- PowerShell doesn't support `mkdir dir1 dir2 dir3` syntax
- Use: `mkdir dir1; mkdir dir2; mkdir dir3`
- Or use Command Prompt: `md dir1 dir2 dir3`

node_modules

- Always gitignored (50,000+ files)
- Regenerate with `npm install`
- Never commit to GitHub

PostgreSQL

- Service must be running
- Check with: Services app or `services.msc`

- Database: `stattrack`
- User: `postgres`
- Port: `5432`

API-Football Rate Limits

- Free tier: 100 requests/day
- Resets at midnight UTC
- Cache aggressively to avoid hitting limit
- Consider upgrading to paid tier for production

Project Documentation Reference

Main Project Plan

See: `football-stat-tracker-project-plan.md`

- Complete 12-week roadmap
- Full database schema with explanations
- All API endpoints specifications
- Deployment strategy
- Testing approach

React Setup Guide

- Vite + React + TypeScript
- TailwindCSS configuration
- shadcn/ui component library
- React Query for data fetching

Environment Setup Checklist

- [x] Node.js 20 LTS installed
- [x] PostgreSQL 18 installed and running
- [x] VS Code installed with extensions
- [x] Git configured with GitHub
- [x] Backend dependencies installed
- [x] Database created (`stattrack`)
- [x] Prisma schema created and migrated
- [x] Environment variables configured
- [x] API-Football key obtained
- [x] Server running successfully

Quick Start Guide for Claude Code

When you start Claude Code, you can say:

```
"I'm working on a football stat tracker project. The context is in project-
context.md.
I've completed the backend foundation and database setup.

Next, I need to:
1. Create the Football API service to call API-Football
2. Create player search endpoint
3. Create player profile endpoint

Help me create the Football API service first. Use TypeScript, handle rate
limiting,
and implement basic caching."
```

Or more simply:

```
"Read project-context.md and help me continue from where I left off.
Start with creating the Football API service."
```

Resources and Links

Documentation

- Express.js: https://expressjs.com/
- Prisma: https://www.prisma.io/docs
- TypeScript: https://www.typescriptlang.org/docs
- PostgreSQL: https://www.postgresql.org/docs
- API-Football: https://www.api-football.com/documentation-v3

Project Links

- GitHub Repo: https://github.com/Lancem238/football-stat-tracker
- API-Football Dashboard: https://rapidapi.com/api-sports/api/api-football
- RapidAPI Account: https://rapidapi.com/

Community

- Stack Overflow: For debugging
- Discord: React, Node.js communities
- Reddit: r/node, r/reactjs, r/typescript

Success Metrics

Current Progress: ~20% Complete

- ☑ Environment setup (Week 0)
- ☑ Backend foundation (Week 1)

- 🔄 API integration (Week 2) ← **YOU ARE HERE**
- ⏳ Frontend setup (Week 3-4)
- ⏳ Core features (Week 5-6)
- ⏳ User features (Week 7-8)
- ⏳ Advanced features (Week 9-10)
- ⏳ Testing & deployment (Week 11-12)

Next Milestone

Complete API endpoints for player search and profile

- Player search working
- Player profile displaying data
- Basic error handling
- API caching implemented

Tips for Working with Claude Code

1. **Be Specific:** "Create a player search endpoint in Express with TypeScript" is better than "make an API"
2. **Reference Files:** "Update src/routes/players.ts to add a new endpoint" helps Claude find the right file
3. **Ask for Explanations:** "Explain why we're using Prisma instead of raw SQL" helps you learn
4. **Incremental Changes:** Build one feature at a time, test it, then move to the next
5. **Use Project Context:** Start conversations with "Based on the project context..." so Claude knows where you are
6. **Git Commits:** After each working feature, commit your changes
7. **Test Frequently:** Run `npm run dev` and test endpoints as you build them

---

Final Notes

You're doing great! You've successfully:

- Set up a professional development environment
- Created a solid database schema
- Got your backend server running
- Connected to PostgreSQL
- Integrated with Prisma ORM

The foundation is solid. Now it's time to build the actual features!

Next Session Goals:

1. Create Football API service
2. Implement player search
3. Test with real API-Football data

4. Add basic error handling

Remember:

- Take it one step at a time
- Test as you go
- Commit working code
- Ask questions when stuck
- Have fun building! ⚽ 🚀

Project Start Date: February 26, 2026
Current Phase: Backend API Development (Week 2)
Status: Active Development
Developer: Lancem238

Good luck with Claude Code! 🎉